

卫星导航算法与程序设计 2

成果总结报告

姓名：李磊

学号：2023302143041

年级：2023 级

班级：智能导航试验班二班

2025 年 9 月 8 日 至 2025 年 11 月 2 日

目录

1. 程序设计概述	2
1.1. 软件功能概述	2
1.2. 软件接口和编程语言	2
1.3. 完成情况	3
2. 算法和程序设计	3
2.1. 时间同步	3
2.2. 站间单差	5
2.3. 基准星选取	8
2.4. 相对定位	10
2.5. 模糊度固定	26
3. 结果精度分析	30
3.1. 数据文件说明	30
3.2. 定位结果	31
3.3. 定位误差分析	32
4. 体会与心得	38

1. 程序设计概述

1.1. 软件功能概述

基于标准 C++ 语言，开发基于 PC 平台的 RTK 相对定位程序，实现从网络实时获取基准站和流动站的 GPS+BDS 双频观测数据，通过相对定位数据处理，获得站间基线向量和流动站当前的高精度位置同时进行精度统计。

软件总体设计为五个模块，分别为时间同步模块、站间单差观测值计算模块、基准星选取模块、相对定位模块、模糊度固定模块。软件总体设计流程图如下：

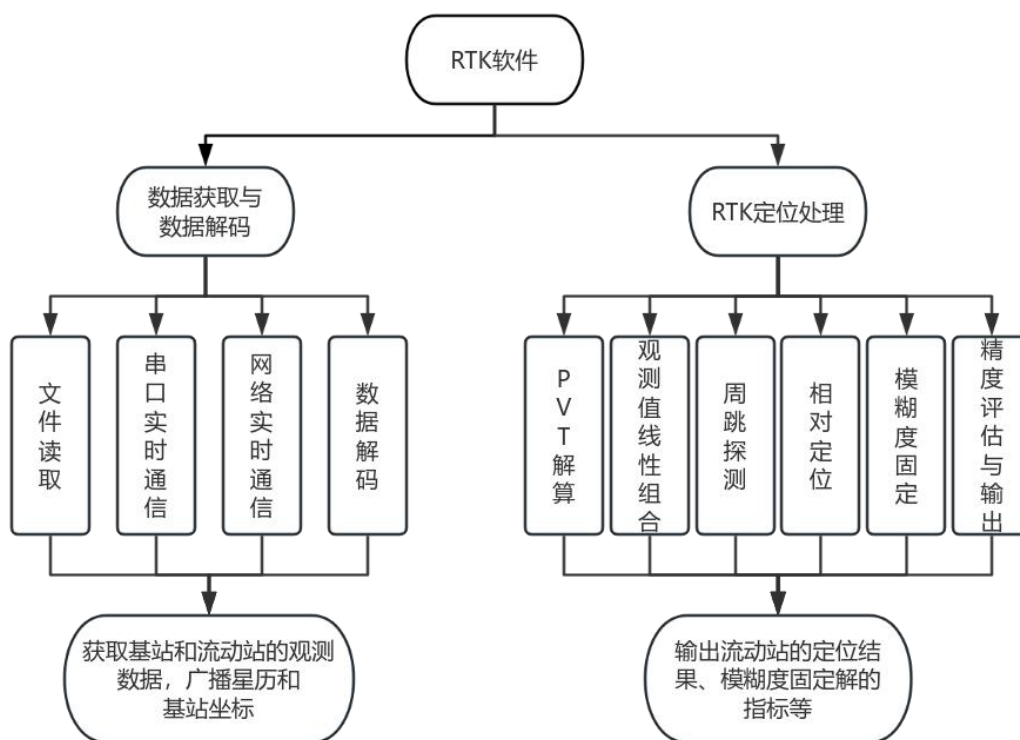


图 1 定位软件总体设计流程图

1.2. 软件接口和编程语言

本程序包含两个输入接口：文件模式，支持读取原始二进制数据

文件进行解算；实时模式，通过 TCP 协议连接到指定服务器（IP：47.114.134.129，端口：7190；IP：8.148.22.229，端口：7002），实时接受 GNSS 数据流。

主要通过两个接口输入：控制台实时显示，在控制台面板中输出定位结果；CSV 文件输出，将完整的定位结果输出到 csv 文本格式文件中。

软件设计主要采用 C/C++ 语言实现算法与程序设计，使用 matlab 进行定位结果的精度分析。

1.3. 完成情况

在课程最后完成了对实时定位程序的实现，能够获取实时的定位结果和模糊度固定指标等信息。实时定位情况如下图所示：

2. 算法和程序设计

2.1. 时间同步

由于单点定位的程序设计中只考虑到了单站的观测值和星历数据，解码时只关注伪距的数据质量。而在 RTK 相对定位中需要同时获取基站和流动站的观测值以及星历数据，且还需要关注载波相位观测值的数据质量（如是否存在半周标记，相位是否失锁等）。由于基站数据通信存在延迟，而 RTK 相对定位需要基站数据和流动站数据的时间严格对齐，所以需要设计时间同步函数。我们保持数据解码的代码相同，在两个测站数据解码后输出到对应的结构体，因此需要设计 RTK 定位数据结构体。

2.1 代码

RTK 结构体设计

```
/*RTK 定位数据定义*/
struct rtkdata
{
    EPOCHOBSDATA rover_obs, base_obs;//基站和流动站数据
    GPSEPHREC geph[MAXGPSNUM], beph[MAXBDSNUM];//星历数据
    SDEPOCHOBS SdObs;//单观测值
    DDCOBS DDObs;//双差相关数据
    POSRES basepos, roverpos;//基站和流动站定位结果
};
```

2.1 代码

时间同步函数

```
int TimeSynch(FILE* roverfile, FILE* basefile, rtkdata*data)//时间同步函数
{
    double dt;
    int lenr_base=0, lenr_rover=0;//实际接收数据长度
    static int lenD_base = 0, lenD_rover = 0;//剩余长度
    static unsigned char buff_base[MAXRAWLEN], buff_rover[MAXRAWLEN];//最大缓冲区
    while (!feof(roverfile)) //读取流动站数据
    {
        if ((lenr_rover = fread(buff_rover + lenD_rover, sizeof(unsigned char), MAXRAWLEN - lenD_rover, roverfile)) < MAXRAWLEN - lenD_rover)return -1;
        lenD_rover += lenr_rover;
        if (DecodeNovOem7Dat(buff_rover, lenD_rover, &data->rover_obs, data->geph, data->beph, &data->roverpos) == 1)break;
    }
    //比较流动站观测时刻和当前基站观测时刻
    dt = (data->rover_obs.Time.week - data->base_obs.Time.week) * 604800 + data->rover_obs.Time.secofweek - data->base_obs.Time.secofweek;
    if (fabs(dt) < 1)return 1; //若不在限差范围内
    while (!feof(basefile))
    {
        if ((lenr_base = fread(buff_base + lenD_base, sizeof(unsigned char), MAXRAWLEN - lenD_base, basefile)) < MAXRAWLEN - lenD_base)return -1;
        lenD_base += lenr_base;
        if (DecodeNovOem7Dat(buff_base, lenD_base, &data->base_obs, data->geph, data->beph, &data->basepos) == 1)
        {
            dt = (data->rover_obs.Time.week - data->base_obs.Time.week) * 604800 + data->rover_obs.Time.secofweek - data->base_obs.Time.secofweek;
            if (fabs(dt) < 1)return 1;
            else if (dt < 0)return 0;
        }
    }
}
```

```

else;
}
}
}

```

该函数首先获取流动站的观测数据，若不成功，继续获取；若文件结束，返回文件结束标记；若成功获取，得到观测时刻。接着将流动站观测时刻与当前基站观测时刻进行比较，在限差范围内，则认为时间同步成功，返回值 1。倘若观测时刻差不在限差范围内，重新获取基站观测时刻，两站的观测时间求差，在限差范围内，则时间同步成功，返回 1；若不在限差范围内，且基站时间在后，返回 0；若基站时间在前，则循环获取基站数据，直到同步成功。

2.2. 站间单差

在相对定位中，为了消除和削弱各种误差对定位的影响，通常采用组合观测值来进行定位，在本软件中，采用双差观测值来进行参数估计。因此首先需要对观测值进行站间单差，便于后续进行双差观测值的构建、观测方程的线性化以及粗差和周跳探测等步骤的进行。

基站观测方程如下：

$$\begin{aligned}
 P_{B,f}^{i_G} &= \rho_B^{i_G} + dt_B^G - c \times dt^{i_G} + I_{B,f}^{i_G} + T_B^{i_G} + p_{B,f}^G - p_f^{i_G} + v_{B,f}^{i_G} \\
 L_{B,f}^{i_G} &= \rho_B^{i_G} + dt_B^G - c \times dt^{i_G} - I_{B,f}^{i_G} + T_B^{i_G} + \lambda_f^G (\delta_{B,f}^G - \delta_f^{i_G} + N_{B,f}^{i_G}) + \varepsilon_{B,f}^{i_G}
 \end{aligned} \tag{1}$$

流动站观测方程如下：

$$\begin{aligned}
 P_{R,f}^{i_G} &= \rho_R^{i_G} + dt_R^G - c \times dt^{i_G} + I_{R,f}^{i_G} + T_R^{i_G} + p_{R,f}^G - p_f^{i_G} + v_{R,f}^{i_G} \\
 L_{R,f}^{i_G} &= \rho_R^{i_G} + dt_R^G - c \times dt^{i_G} - I_{R,f}^{i_G} + T_R^{i_G} + \lambda_f^G (\delta_{R,f}^G - \delta_f^{i_G} + N_{R,f}^{i_G}) + \varepsilon_{R,f}^{i_G}
 \end{aligned} \tag{2}$$

对基站和流动站数据中相同卫星，相同频率，相同类型观测值进行站间求差，能够消除卫星钟差、伪距硬件延迟、初始相位延迟和相位硬件延迟；同时利用电离层延迟、对流层延迟和卫星轨道误差这类误差

的强空间相关性，站间单差可以削弱这些误差的影响。求差后的观测方程如下：

$$\begin{aligned}
 P_{BR,f}^{i_G} &= P_{R,f}^{i_G} - P_{B,f}^{i_G} = \rho_R^{i_G} - \rho_B^{i_G} + dt_R^G - dt_B^G + I_{R,f}^{i_G} - I_{B,f}^{i_G} + T_R^{i_G} - T_B^{i_G} \\
 &\quad + p_{R,f}^G - p_{B,f}^G + v_{BR,f}^{i_G} \\
 L_{BR,f}^{i_G} &= L_{R,f}^{i_G} - L_{B,f}^{i_G} = \rho_R^{i_G} - \rho_B^{i_G} + dt_R^G - dt_B^G + I_{R,f}^{i_G} - I_{B,f}^{i_G} + T_R^{i_G} - T_B^{i_G} \\
 &\quad + \lambda_f^G \cdot (\delta_{R,f}^G - \delta_{B,f}^G + N_{R,f}^{i_G} - N_{B,f}^{i_G}) + \varepsilon_{BR,f}^{i_G}
 \end{aligned} \tag{3}$$

利用构建的单差观测方程，可以进行周跳探测和半周标记检验。

通过检测接收机质量数据 **Locktime**，来判断历元间相位数据的连续性，没有发生中断时，若 $Locktime(k+1) \geq Locktime(k)$ ，数据正常。由于 **MW** 组合进行周跳探测时无法检测出小于一周的相位偏差，因此需要检测 **Parity** 来判断数据是否存在半周。周跳探测函数可以直接使用单点定位软件中的周跳探测函数。

2.2 代码
站间单差观测值结构体定义
<pre> struct SDSATOBS{ short prn; GNSSSys System; bool Valid; double dP[2], dL[2]; short nBas, nRov;//储存单差观测值对应的基准站和流动站的数值索引号 SDSATOBS() { prn = 0; nBas = -1; nRov = -1; System = UNKS; dP[0] = dP[1] = dL[0] = dL[1] = 0.0; Valid =0; }; </pre>

2.2 代码
站间单差程序设计
<pre> void FormSDEpochObs(EPOCHOBSDATA* EpkB, EPOCHOBSDATA* EpkR, SDEPOCHOBS* SDObs) { // 1. 参数有效性检查 </pre>

```

if (!EpkB || !EpkR || !SDObs) {
    return;
}

// 2. 数组边界检查
if (!SDObs->SdSatObs) {
    return;
}

// 重置计数器
SDObs->SatNum = 0;
for (int i = 0; i < EpkR->Satnum; i++)
{
    if (!EpkR->SatObs[i].Valid) continue;
    for (int j = 0; j < EpkB->Satnum; j++)
    {
        if (!EpkB->SatObs[j].Valid) continue;
        // 对同类型和同频率的观测值求差
        if (EpkR->SatObs[i].Prn == EpkB->SatObs[j].Prn &&
            EpkR->SatObs[i].system == EpkB->SatObs[j].system)
        {
            // 按照第一段代码逻辑添加有效性检查
            // 检查锁定时长等观测数据质量指标
            if (EpkR->SatObs[i].LockTime[0] < 6 || EpkR->SatObs[i].LockTime[1] < 6 ||
                EpkB->SatObs[j].LockTime[0] < 6 || EpkB->SatObs[j].LockTime[1] < 6) {
                continue; // 锁定时长不满足要求, 跳过
            }
            // 计算单差观测值
            for (int k = 0; k < 2; k++)
            {
                SDObs->SdSatObs[SDObs->SatNum].dP[k] = EpkR->SatObs[i].P[k] - EpkB->SatObs[j].P[k];
                SDObs->SdSatObs[SDObs->SatNum].dL[k] = EpkR->SatObs[i].L[k] - EpkB->SatObs[j].L[k];
            }
            // 保存卫星信息
            SDObs->SdSatObs[SDObs->SatNum].prn = EpkR->SatObs[i].Prn;
            SDObs->SdSatObs[SDObs->SatNum].System = EpkR->SatObs[i].system;
            SDObs->SdSatObs[SDObs->SatNum].nBas = j;
            SDObs->SdSatObs[SDObs->SatNum].nRov = i;
            // 增加计数
            SDObs->SatNum++;
            break;
        }
    }
}

// 设置时间
SDObs->Time = EpkR->Time;

```


}

该函数根据流动站卫星数进行循环，对于每颗流动站的卫星，首先检查卫星号和系统号是否正常，卫星观测值是否完整，如果卫星不正常则 **continue**，处理下一颗卫星；然后在基站观测值中查找与该流动站相同的卫星，如果未找到，就 **continue**，处理下一颗流动站卫星；对于同类型和同频率的观测值求差并保存，同时保存索引号、卫星号和系统号等相关信息，累加单差观测值卫星数量；最后，循环结束，对单差观测值的观测时刻和卫星数量赋值。

2.3. 基准星选取

由式(3)可以得到 j_G 卫星的站间单差方程：

$$\begin{aligned}
 P_{BR,f}^{j_G} &= P_{R,f}^{j_G} - P_{B,f}^{j_G} = \rho_R^{j_G} - \rho_B^{j_G} + dt_R^G - dt_B^G + I_{R,f}^{j_G} - I_{B,f}^{j_G} + T_R^{j_G} - T_B^{j_G} \\
 &\quad + p_{R,f}^G - p_{B,f}^G + v_{BR,f}^{j_G} \\
 L_{BR,f}^{j_G} &= L_{R,f}^{j_G} - L_{B,f}^{j_G} = \rho_R^{j_G} - \rho_B^{j_G} + dt_R^G - dt_B^G + I_{R,f}^{j_G} - I_{B,f}^{j_G} + T_R^{j_G} - T_B^{j_G} \\
 &\quad + \lambda_f^G \cdot (\delta_{R,f}^G - \delta_{B,f}^G + N_{R,f}^{j_G} - N_{B,f}^{j_G}) + \varepsilon_{BR,f}^{j_G}
 \end{aligned} \tag{4}$$

对各卫星导航系统，分别选取一颗观测值数据质量高的卫星作为参考星，其他卫星的单差观测值与该参考星对应的单差观测值求差。以 i_G 卫星为参考星，联立式(3)(4)，可得双差观测方程：

$$\begin{aligned}
 P_{BR,f}^{i_G j_G} &= P_{BR,f}^{j_G} - P_{BR,f}^{i_G} = \rho_R^{j_G} - \rho_B^{j_G} - \rho_R^{i_G} + \rho_B^{i_G} + I_{R,f}^{j_G} - I_{B,f}^{j_G} - I_{R,f}^{i_G} + I_{B,f}^{i_G} \\
 &\quad + T_R^{j_G} - T_B^{j_G} - T_R^{i_G} + T_B^{i_G} + v_{BR,f}^{i_G j_G} \\
 &\approx \rho_R^{j_G} - \rho_B^{j_G} - \rho_R^{i_G} + \rho_B^{i_G} + v_{BR,f}^{i_G j_G} \\
 L_{BR,f}^{i_G j_G} &= L_{BR,f}^{j_G} - L_{BR,f}^{i_G} = \rho_R^{j_G} - \rho_B^{j_G} - \rho_R^{i_G} + \rho_B^{i_G} + I_{R,f}^{j_G} - I_{B,f}^{j_G} - I_{R,f}^{i_G} + I_{B,f}^{i_G} \\
 &\quad + T_R^{j_G} - T_B^{j_G} - T_R^{i_G} + T_B^{i_G} + \lambda_f^G \cdot (N_{R,f}^{j_G} - N_{B,f}^{j_G} - N_{R,f}^{i_G} + N_{B,f}^{i_G}) + \varepsilon_{BR,f}^{i_G j_G} \\
 &\approx \rho_R^{j_G} - \rho_B^{j_G} - \rho_R^{i_G} + \rho_B^{i_G} + \lambda_f^G \cdot N_{BR,f}^{i_G j_G} + \varepsilon_{BR,f}^{i_G j_G}
 \end{aligned} \tag{5}$$

这样的双差观测值消除了卫星钟差、卫星端伪距硬件延迟、初相和相位延迟，消除了接收机钟差、伪距硬件延迟、初相和相位硬件延

迟。同时，双差模糊度具有整数特性。

选取参考星时，一方面要保证卫星的观测数据没有“差错”，即没有粗差和周跳，卫星星历正常，卫星位置能成功计算；另一方面，要保证卫星观测数据的质量，软件中选取卫星高度角和信号载噪比作为质量评估的指标，用来选取最佳参考星。

2.3 代码

基准星选取程序设计

```
void DetRefSat(const EPOCHOBSDATA* epkA, const EPOCHOBSDATA* epkB, SDEPOCHOBS* SDObs, DD
COBS* DDObs)
{
    int i, j, n;
    double Sum[2] = { 0.0 }, MaxSum[2] = { 0.0 };
    int RefPrn[2] = { -1 }, RefIndex[2] = { -1 };
    for (int i = 0; i < SDObs->SatNum; i++)
    {
        if (!SDObs->SdSatObs[i].Valid || !epkA->SatPVT[SDObs->SdSatObs[i].nBas].Valid || !epkB->SatPVT[S
DObs->SdSatObs[i].nRov].Valid) continue;
        if (epkA->SatObs[SDObs->SdSatObs[i].nBas].LockTime[0] < 6 || epkA->SatObs[SDObs->SdSatObs[i
].nBas].LockTime[1] < 6) continue;
        n = SDObs->SdSatObs[i].System == GPS ? 0 : 1;
        Sum[n] = epkB->SatPVT[SDObs->SdSatObs[i].nRov].Elevation + epkA->SatObs[SDObs->SdSatObs[
i].nBas].cn0[0] + epkA->SatObs[SDObs->SdSatObs[i].nBas].cn0[1] +
            epkB->SatObs[SDObs->SdSatObs[i].nRov].cn0[0] + epkB->SatObs[SDObs->SdSatObs[i].nRov].
cn0[1]; //质量评估指标
        if (Sum[n] > MaxSum[n]) {
            MaxSum[n] = Sum[n];
            RefPrn[n] = SDObs->SdSatObs[i].prn;
            RefIndex[n] = i;
        }
    }
    for (i = 0; i < 2; i++) {
        if (MaxSum[i] < 150.0) DDObs->RefPos[i] = -1;
        else
        {
            DDObs->RefPos[i] = RefIndex[i]; //参考星索引
            DDObs->RefPrn[i] = RefPrn[i]; //参考星 prn 号
        }
    }
}
```

2.4. 相对定位

(1) 基于最小二乘原理的相对定位算法

在短基线情况下，如式(5)所示，可以忽略电离层和对流层残余误差的影响，对于原始观测值的双差而言，则保留了模糊度的整数特性，未知参数一般包括基线向量和模糊度参数。双差观测方程如下：

$$\begin{aligned} P_{BR,f}^{i_G j_G} &= P_{BR,f}^{j_G} - P_{BR,f}^{i_G} = \rho_R^{j_G} - \rho_B^{j_G} - \rho_R^{i_G} + \rho_B^{i_G} + v_{BR,f}^{i_G j_G} \\ L_{BR,f}^{i_G j_G} &= L_{BR,f}^{j_G} - L_{BR,f}^{i_G} = \rho_R^{j_G} - \rho_B^{j_G} - \rho_R^{i_G} + \rho_B^{i_G} + \lambda_f^G \cdot N_{BR,f}^{i_G j_G} + \varepsilon_{BR,f}^{i_G j_G} \end{aligned} \quad (6)$$

以解码出的基站参考坐标（倘若没有该信息，就使用 SPP 的定位结果）和流动站 SPP 的定位结果对式(6)线性化，得到线性化后的方程如下：

$$\begin{aligned} \rho_{BR}^{i_G j_G} &= \rho_{R_0}^{j_G}(t) + \frac{X_{R_0} - X^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} \Delta X_R + \frac{Y_{R_0} - Y^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} \Delta Y_R + \frac{Z_{R_0} - Z^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} \Delta Z_R \\ &\quad - \left(\rho_{B_0}^{j_G}(t) + \frac{X_{B_0} - X^{j_G}(t)}{\rho_{B_0}^{j_G}(t)} \Delta X_B + \frac{Y_{B_0} - Y^{j_G}(t)}{\rho_{B_0}^{j_G}(t)} \Delta Y_B + \frac{Z_{B_0} - Z^{j_G}(t)}{\rho_{B_0}^{j_G}(t)} \Delta Z_B \right) \\ &\quad - \left(\rho_{R_0}^{i_G}(t) + \frac{X_{R_0} - X^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \Delta X_R + \frac{Y_{R_0} - Y^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \Delta Y_R + \frac{Z_{R_0} - Z^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \Delta Z_R \right) \\ &\quad - \left(\rho_{B_0}^{i_G}(t) + \frac{X_{B_0} - X^{i_G}(t)}{\rho_{B_0}^{i_G}(t)} \Delta X_B + \frac{Y_{B_0} - Y^{i_G}(t)}{\rho_{B_0}^{i_G}(t)} \Delta Y_B + \frac{Z_{B_0} - Z^{i_G}(t)}{\rho_{B_0}^{i_G}(t)} \Delta Z_B \right) \end{aligned} \quad (7)$$

基站点坐标是已知的 X_{B_0} ， $\Delta X_B = \Delta Y_B = \Delta Z_B = 0$ ，流动站初始坐标为 X_{R_0} ，则式(7)可以变化为下面的形式：

$$\begin{aligned}
 \rho_{BR}^{i_G j_G} &= \rho_{B_0}^{i_G}(t) - \rho_{B_0}^{j_G}(t) \\
 &+ \left(\rho_{R_0}^{j_G}(t) + \frac{X_{R_0} - X^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} \Delta X_R + \frac{Y_{R_0} - Y^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} \Delta Y_R + \frac{Z_{R_0} - Z^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} \Delta Z_R \right) \\
 &- \left(\rho_{R_0}^{i_G}(t) + \frac{X_{R_0} - X^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \Delta X_R + \frac{Y_{R_0} - Y^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \Delta Y_R + \frac{Z_{R_0} - Z^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \Delta Z_R \right) \quad (8) \\
 &= \rho_{R_0}^{j_G}(t) - \rho_{R_0}^{i_G}(t) - \rho_{B_0}^{j_G}(t) + \rho_{B_0}^{i_G}(t) + \left(\frac{X_{R_0} - X^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{X_{R_0} - X^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta X_R \\
 &+ \left(\frac{Y_{R_0} - Y^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{Y_{R_0} - Y^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta Y_R + \left(\frac{Z_{R_0} - Z^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{Z_{R_0} - Z^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta Z_R
 \end{aligned}$$

以流动站初始坐标的改正数和双差模糊度为待估参数，线性化后的观测方程为：

$$\begin{aligned}
 P_{BR,f}^{i_G j_G} &= \rho_{B_0 R_0}^{i_G j_G} + \left(\frac{X_{R_0} - X^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{X_{R_0} - X^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta X_R + \left(\frac{Y_{R_0} - Y^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{Y_{R_0} - Y^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta Y_R \\
 &+ \left(\frac{Z_{R_0} - Z^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{Z_{R_0} - Z^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta Z_R + v_{BR,f}^{i_G j_G} \quad (9) \\
 I_{BR,f}^{i_G j_G} &= \rho_{B_0 R_0}^{i_G j_G} + \left(\frac{X_{R_0} - X^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{X_{R_0} - X^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta X_R + \left(\frac{Y_{R_0} - Y^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{Y_{R_0} - Y^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta Y_R \\
 &+ \left(\frac{Z_{R_0} - Z^{j_G}(t)}{\rho_{R_0}^{j_G}(t)} - \frac{Z_{R_0} - Z^{i_G}(t)}{\rho_{R_0}^{i_G}(t)} \right) \Delta Z_R + \lambda_f^G \cdot N_{BR,f}^{i_G j_G} + \varepsilon_{BR,f}^{i_G j_G}
 \end{aligned}$$

对于双频双系统的观测方程，若参考卫星 i_G 为 G_1 和 B_1 ，观测卫星为 $G_2 \dots, B_2 \dots$ ，将方向余弦表示成下面的形式：

$$\begin{aligned}
 l_{R_0}^{G_1 G_2}(t) &= \frac{X_{R_0} - X^{G_2}(t)}{\rho_{R_0}^{G_2}(t)} - \frac{X_{R_0} - X^{G_1}(t)}{\rho_{R_0}^{G_1}(t)} \\
 m_{R_0}^{G_1 G_2}(t) &= \frac{Y_{R_0} - Y^{G_2}(t)}{\rho_{R_0}^{G_2}(t)} - \frac{Y_{R_0} - Y^{G_1}(t)}{\rho_{R_0}^{G_1}(t)} \\
 n_{R_0}^{G_1 G_2}(t) &= \frac{Z_{R_0} - Z^{G_2}(t)}{\rho_{R_0}^{G_2}(t)} - \frac{Z_{R_0} - Z^{G_1}(t)}{\rho_{R_0}^{G_1}(t)} \quad (10)
 \end{aligned}$$

则观测方程可以表示为：

$$\begin{aligned}
 P_{BR,f_1}^{G_1G_2} &= \rho_{B_0R_0}^{G_1G_2} + l_{R_0}^{G_1G_2}(t) \Delta X_R + m_{R_0}^{G_1G_2}(t) \Delta Y_R + n_{R_0}^{G_1G_2}(t) \Delta Z_R + \hat{v}_{BR} \\
 P_{BR,f_2}^{G_1G_2} &= \rho_{B_0R_0}^{G_1G_2} + l_{R_0}^{G_1G_2}(t) \Delta X_R + m_{R_0}^{G_1G_2}(t) \Delta Y_R + n_{R_0}^{G_1G_2}(t) \Delta Z_R + \hat{v}_{BR} \\
 L_{BR,f_1}^{G_1G_2} &= \rho_{B_0R_0}^{G_1G_2} + l_{R_0}^{G_1G_2}(t) \Delta X_R + m_{R_0}^{G_1G_2}(t) \Delta Y_R + n_{R_0}^{G_1G_2}(t) \Delta Z_R + \lambda_{f_1}^G \cdot N_{BR,f_1}^{G_1G_2} + \mathcal{E}_{BR}^G \\
 L_{BR,f_2}^{G_1G_2} &= \rho_{B_0R_0}^{G_1G_2} + l_{R_0}^{G_1G_2}(t) \Delta X_R + m_{R_0}^{G_1G_2}(t) \Delta Y_R + n_{R_0}^{G_1G_2}(t) \Delta Z_R + \lambda_{f_2}^G \cdot N_{BR,f_2}^{G_1G_2} + \mathcal{E}_{BR}^G \\
 &\dots \\
 P_{BR,f_1}^{B_1B_2} &= \rho_{B_0R_0}^{B_1B_2} + l_{R_0}^{B_1B_2}(t) \Delta X_R + m_{R_0}^{B_1B_2}(t) \Delta Y_R + n_{R_0}^{B_1B_2}(t) \Delta Z_R + \hat{v}_{BR} \\
 P_{BR,f_3}^{B_1B_2} &= \rho_{B_0R_0}^{B_1B_2} + l_{R_0}^{B_1B_2}(t) \Delta X_R + m_{R_0}^{B_1B_2}(t) \Delta Y_R + n_{R_0}^{B_1B_2}(t) \Delta Z_R + \hat{v}_{BR} \\
 L_{BR,f_1}^{B_1B_2} &= \rho_{B_0R_0}^{B_1B_2} + l_{R_0}^{B_1B_2}(t) \Delta X_R + m_{R_0}^{B_1B_2}(t) \Delta Y_R + n_{R_0}^{B_1B_2}(t) \Delta Z_R + \lambda_{f_1}^B \cdot N_{BR,f_1}^{B_1B_2} + \mathcal{E}_{BR}^B \\
 L_{BR,f_3}^{B_1B_2} &= \rho_{B_0R_0}^{B_1B_2} + l_{R_0}^{B_1B_2}(t) \Delta X_R + m_{R_0}^{B_1B_2}(t) \Delta Y_R + n_{R_0}^{B_1B_2}(t) \Delta Z_R + \lambda_{f_3}^B \cdot N_{BR,f_3}^{B_1B_2} + \mathcal{E}_{BR}^B \\
 &\dots
 \end{aligned} \tag{11}$$

将观测方程写成矩阵形式时，系数矩阵如下：

$$B(t) = \begin{bmatrix}
 l_{R_0}^{G_1G_2} & m_{R_0}^{G_1G_2} & n_{R_0}^{G_1G_2} & 0 & 0 & \dots & 0 & 0 & \dots \\
 l_{R_0}^{G_1G_2} & m_{R_0}^{G_1G_2} & n_{R_0}^{G_1G_2} & 0 & 0 & \dots & 0 & 0 & \dots \\
 l_{R_0}^{G_1G_2} & m_{R_0}^{G_1G_2} & n_{R_0}^{G_1G_2} & \lambda_{f_1}^G & 0 & \dots & 0 & 0 & \dots \\
 l_{R_0}^{G_1G_2} & m_{R_0}^{G_1G_2} & n_{R_0}^{G_1G_2} & 0 & \lambda_{f_2}^G & \dots & \dots & \dots & \dots \\
 \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\
 l_{R_0}^{B_1B_2} & m_{R_0}^{B_1B_2} & n_{R_0}^{B_1B_2} & 0 & 0 & \dots & 0 & 0 & \dots \\
 l_{R_0}^{B_1B_2} & m_{R_0}^{B_1B_2} & n_{R_0}^{B_1B_2} & 0 & 0 & \dots & 0 & 0 & \dots \\
 l_{R_0}^{B_1B_2} & m_{R_0}^{B_1B_2} & n_{R_0}^{B_1B_2} & 0 & 0 & \dots & \lambda_{f_1}^B & 0 & \dots \\
 l_{R_0}^{B_1B_2} & m_{R_0}^{B_1B_2} & n_{R_0}^{B_1B_2} & 0 & 0 & \dots & 0 & \lambda_{f_3}^B & \dots \\
 \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots
 \end{bmatrix} \tag{12}$$

则待求解变量为：

$$X(t) = [\Delta X_R \quad \Delta Y_R \quad \Delta Z_R \quad N_{BR,f_1}^{G_1G_2} \quad N_{BR,f_2}^{G_1G_2} \quad \dots \quad N_{BR,f_1}^{B_1B_2} \quad N_{BR,f_3}^{B_1B_2} \quad \dots]^T \tag{13}$$

定义观测值残差向量：

$$W(t) = [P_{BR,f_1}^{G_1G_2} - \rho_{B_0R_0}^{G_1G_2} \quad \dots \quad L_{BR,f_1}^{G_1G_2} - \rho_{B_0R_0}^{G_1G_2} - \lambda_{f_1}^G \cdot N_{BR,f_1}^{G_1G_2} \quad \dots \quad P_{BR,f_1}^{B_1B_2} - \rho_{B_0R_0}^{B_1B_2} \quad \dots \quad L_{BR,f_1}^{B_1B_2} - \rho_{B_0R_0}^{B_1B_2} - \lambda_{f_1}^B \cdot N_{BR,f_1}^{B_1B_2} \quad \dots]^T \tag{14}$$

则观测方程可以改写为下面的形式：

$$W(t) = B(t)X(t) + v \tag{15}$$

为了获取最小二乘解算的权阵，下面推导观测值之间的相关性。

1. 单差观测值之间的相关性（站间单差）

单差观测方程为

$$\begin{aligned}\Phi_{AB}^j(t) &= \Phi_B^j(t) - \Phi_A^j(t) \\ \Phi_{AB}^k(t) &= \Phi_B^k(t) - \Phi_A^k(t)\end{aligned}\quad (16)$$

单差矩阵为 $SD = \begin{bmatrix} \Phi_{AB}^j(t) \\ \Phi_{AB}^k(t) \end{bmatrix}$ ，观测值方程为

$$\Phi = [\Phi_A^j(t) \quad \Phi_B^j(t) \quad \Phi_A^k(t) \quad \Phi_B^k(t)]^T, \text{ 系数矩阵为 } C = \begin{bmatrix} -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix}, \text{ 单}$$

差观测方程可写成 $SD = C\Phi$ ，由误差传播定律，可以求出单差观测值的协方差矩阵：

$$\text{cov}(SD) = C \text{cov}(\Phi) C^T = \sigma^2 C C^T = 2\sigma^2 I \quad (17)$$

式(17)表明这个协方差矩阵是一个对角矩阵，即单差观测值之间是数学不相关的。

2. 双差观测值间的相关性（两个测站，两颗卫星一组）

双差观测值方程为：

$$\begin{aligned}\Phi_{AB}^{jk}(t) &= \Phi_{AB}^k(t) - \Phi_{AB}^j(t) \\ \Phi_{AB}^{jl}(t) &= \Phi_{AB}^l(t) - \Phi_{AB}^j(t)\end{aligned}\quad (18)$$

双差观测矩阵为 $DD = \begin{bmatrix} \Phi_{AB}^{jk}(t) \\ \Phi_{AB}^{jl}(t) \end{bmatrix}$ ，单差观测值方程为 $SD = \begin{bmatrix} \Phi_{AB}^j(t) \\ \Phi_{AB}^k(t) \\ \Phi_{AB}^l(t) \end{bmatrix}$ ，稀疏

矩阵为 $C = \begin{bmatrix} -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}$ ，双差观测方程可以写成 $DD = CSD$ ，由误差传播

定律，可以求出双差观测值的协方差矩阵：

$$\text{cov}(DD) = C \text{cov}(SD) C^T = 2\sigma^2 \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad (19)$$

则该情况下的权阵为：

$$P(t) = [\text{cov}(DD)]^{-1} = \frac{1}{2\sigma^2} \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \quad (20)$$

据此可知双差观测值之间是数学相关的。

由数学归纳法，可知 n_{DD} 个双差观测值的权阵（针对 GPS 系统的第一频率的相位观测值）为 $P(t) = \frac{1}{2\sigma^2} \frac{1}{n_{DD} + 1} \begin{bmatrix} n_{DD} & -1 & -1 & \dots \\ -1 & n_{DD} & -1 & \dots \\ -1 & \dots & n_{DD} & \dots \\ \dots & \dots & \dots & n_{DD} \end{bmatrix}$ 。倘若同

时 GPS 和 BDS 卫星，且同时考虑伪距和相位观测噪声，设置权阵就会比较复杂。由于相位观测值的精度要比伪距观测值的精度高，一般将伪距噪声设置为 0.3m，将载波相位噪声设置为 0.01m。通过上面的推导不难发现，同一卫星系统间，不同频率和不同种类的双差观测值之间是不相关的；由于参考卫星相同，所以相同频率，相同种类的双差观测值之间是相关的；不同卫星系统之间的双差观测值是不相关的。

权阵设置的代码如下：

2.4 代码
权阵设置程序设计
<pre> // 构建设计矩阵和权阵 for (int i = 0; i < SDObs->SatNum; i++) { // 跳过参考星和不可用的卫星 if (i == ref_idx !SDObs->SdSatObs[i].Valid) continue; // 初始化权阵 int tmp_idx = 0; for (int j = 0; j < SDObs->SatNum; j++) { if (j == DDObs->RefPos[0] j == DDObs->RefPos[1] !SDObs->SdSatObs[j].Valid) continue; if (SDObs->SdSatObs[i].System != SDObs->SdSatObs[j].System) { tmp_idx++; continue; } int ddnum = (SDObs->SdSatObs[i].System == GPS) ? ddgpsnum : ddbdsnum; } } </pre>

```

if (curr_id == tmp_idx)
{
    P(curr_id * 4 + 0, tmp_idx * 4 + 0) = ddnum / (pow(sigma_p, 2) * (ddnum + 1));
    P(curr_id * 4 + 1, tmp_idx * 4 + 1) = ddnum / (pow(sigma_p, 2) * (ddnum + 1));
    P(curr_id * 4 + 2, tmp_idx * 4 + 2) = ddnum / (pow(sigma_l, 2) * (ddnum + 1));
    P(curr_id * 4 + 3, tmp_idx * 4 + 3) = ddnum / (pow(sigma_l, 2) * (ddnum + 1));
}
else
{
    P(curr_id * 4 + 0, tmp_idx * 4 + 0) = -1 / (pow(sigma_p, 2) * (ddnum + 1));
    P(curr_id * 4 + 1, tmp_idx * 4 + 1) = -1 / (pow(sigma_p, 2) * (ddnum + 1));
    P(curr_id * 4 + 2, tmp_idx * 4 + 2) = -1 / (pow(sigma_l, 2) * (ddnum + 1));
    P(curr_id * 4 + 3, tmp_idx * 4 + 3) = -1 / (pow(sigma_l, 2) * (ddnum + 1));
}
tmp_idx++;
}
curr_id++;
}
  
```

为权阵赋值的思路就是依据前面的推导，将对角线和非对角线元素分开赋值，且按照不同的卫星进行分块赋值即可。

至此，对于最小二乘浮点解的数学推导结束，程序设计的思路如下所示：

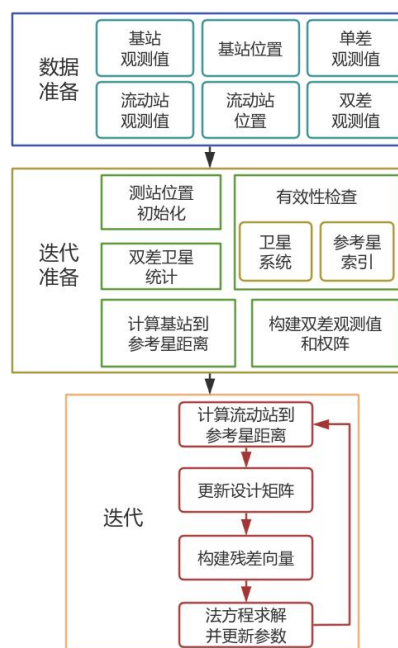


图 2 最小二乘浮点解程序设计流程

首先，设置基站和流动站位置的初值，计算 GPS 和 BDS 的双差卫星数；然后计算基站到所有卫星的几何距离，计算流动站到参考星的几何距离；接着对单差观测值进行循环，线性化双差观测方程，得到设计矩阵和观测值残差向量，同时计算权矩阵；最后解算法方程，更新流动站的位置和双差模糊度参数，若改正数不满足迭代终止条件，则重新计算流动站到参考星的几何距离，进行迭代。若迭代终止，则浮点解计算完成，保存双差浮点模糊度及其协因数矩阵，用于 LAMBDA 模糊度固定。

2.4 代码
最小二乘迭代求解程序设计
<pre> // 最小二乘迭代求解浮点解 int count = 0, rows = 0; int ref_rover[2] = { SDObs->SdSatObs[DDObs->RefPos[0]].nRov, SDObs->SdSatObs[DDObs->RefPos[1]].nRov }; MatrixXd float_Qxx; // 浮点解协方差矩阵 // 检查流动站参考星索引有效性 if (ref_rover[0] < 0 ref_rover[0] >= epkB->Satnum ref_rover[1] < 0 ref_rover[1] >= epkB->Satnum) { return 0; } do{ VectorXd distance_rover2sat(DDObs->Sats); B.setZero(); W.setZero(); rows = 0; for (int i = 0; i < 3; i++) { roverref_sat_PVT_GPS(i, 0) = epkB->SatPVT[ref_rover[0]].SatPos[i]; roverref_sat_PVT_BDS(i, 0) = epkB->SatPVT[ref_rover[1]].SatPos[i]; } double distance_rover2sat_ref[2] = { (X.head(3) - roverref_sat_PVT_GPS).norm(), (X.head(3) - roverref_sat_PVT_BDS).norm() }; for (int i = 0; i < SDObs->SatNum; i++) { </pre>

```

int n = -1;
if (SDObs->SdSatObs[i].System == GPS) n = 0;
else if (SDObs->SdSatObs[i].System == BDS) n = 1;
else continue;
int ref_idx = DDObs->RefPos[n];
int rover_idx = SDObs->SdSatObs[i].nRov;
int base_idx = SDObs->SdSatObs[i].nBas;
// 跳过参考星和不可用卫星
if (i == ref_idx || !SDObs->SdSatObs[i].Valid) continue;
// 流动站到非参考星的距离计算
distance_rover2sat(rows, 0) = sqrt(
    pow(X(0) - epkB->SatPVT[rover_idx].SatPos[0], 2) +
    pow(X(1) - epkB->SatPVT[rover_idx].SatPos[1], 2) +
    pow(X(2) - epkB->SatPVT[rover_idx].SatPos[2], 2)
);
double f1 = 0, f2 = 0;
if (SDObs->SdSatObs[i].System == GPS) {
    f1 = WL1_GPS; f2 = WL2_GPS;
}
else {
    f1 = WL1_BDS; f2 = WL3_BDS;
}
double range_dd = (distance_rover2sat(rows) - distance_rover2sat_ref[n]
    - (distance_base2sat(rows) - distance_base2ref[n]));
W(4 * rows + 0) = N(4 * rows + 0) - range_dd;
W(4 * rows + 1) = N(4 * rows + 1) - range_dd;
W(4 * rows + 2) = N(4 * rows + 2) - range_dd - f1 * X(3 + rows * 2 + 0);
W(4 * rows + 3) = N(4 * rows + 3) - range_dd - f2 * X(3 + rows * 2 + 1);
// 观测方程线性化
// 设计矩阵
double l = (X(0) - epkB->SatPVT[rover_idx].SatPos[0]) / distance_rover2sat(rows) -
    (X(0) - epkB->SatPVT[ref_rover[n]].SatPos[0]) / distance_rover2sat_ref[n];
double m = (X(1) - epkB->SatPVT[rover_idx].SatPos[1]) / distance_rover2sat(rows) -
    (X(1) - epkB->SatPVT[ref_rover[n]].SatPos[1]) / distance_rover2sat_ref[n];
double n_val = (X(2) - epkB->SatPVT[rover_idx].SatPos[2]) / distance_rover2sat(ro
ws) -
    (X(2) - epkB->SatPVT[ref_rover[n]].SatPos[2]) / distance_rover2sat_ref[n];
for (int j = 0; j < 4; j++)
{
    B(4 * rows + j, 0) = l;
    B(4 * rows + j, 1) = m;
    B(4 * rows + j, 2) = n_val;
}

```

```

B(rows * 4 + 2, 3 + rows * 2 + 0) = f1;
B(rows * 4 + 3, 3 + rows * 2 + 1) = f2;
rows++;
}
// 观测方程数不足
if (rows * 4 < total_params || SDObs->SatNum < 4) break;
// 计算法方程和协方差矩阵
MatrixXd N_mat = B.transpose() * P * B;
float_Qxx = N_mat.inverse(); // 保存协方差矩阵用于模糊度固定
delta_X = float_Qxx * B.transpose() * P * W;
if (isnan(delta_X(0)))
{
    cout << "nan detected in delta_X" << endl;
    break;
}
X += delta_X;
count++;
} while (delta_X.norm() > 0.001 && count < 100);
// 保存浮点解
rover->Pos[0] = X(0);
rover->Pos[1] = X(1);
rover->Pos[2] = X(2);

```

(2) 基于卡尔曼滤波的相对定位算法

首先确定待估参数，在相对定位中，我们将流动站的位置速度参数和双差模糊度作为待估参数来构建状态方程：

$$X_k = \Phi_{k,k-1} X_{k-1} + \Gamma_{k-1} w_k \quad (21)$$

线性化后的观测方程可以写成：

$$L_k = H_k X_k + v_k \quad (22)$$

过程噪声的随机模型和观测噪声的随机模型如下：

$$\begin{aligned}
 E(w_k) &= 0, \text{Cov}\{w_k, w_j\} = Q_k \delta_{kj} \\
 E(v_k) &= 0, \text{Cov}\{v_k, v_j\} = R_k \delta_{kj} \\
 \text{Cov}\{w_k, v_j\} &= 0
 \end{aligned} \quad (23)$$

卡尔曼滤波分为两个步骤：时间更新步和测量更新步。时间更新即上一历元的状态向量和方差协方差阵通过状态转移矩阵预测得到当前历元的状态向量和方差协方差阵：

$$\begin{aligned}\hat{X}_{k,k-1} &= \Phi_{k,k-1} \hat{X}_{k-1} \\ P_{k,k-1} &= \Phi_{k,k-1} P_{k-1} \Phi_{k,k-1}^T + \Gamma_{k-1} Q_{k-1} \Gamma_{k-1}^T\end{aligned}\quad (24)$$

测量更新即利用当前历元的观测信息对预测的状态向量及其方差协方差阵进行修正：

$$\begin{aligned}K_k &= P_{k,k-1} H_k^T (H_k P_{k,k-1} H_k^T + R_k)^{-1} \\ \hat{X}_k &= \hat{X}_{k,k-1} + K_k (L_k - H_k \hat{X}_{k,k-1}) \\ P_k &= (I - K_k H_k) P_{k,k-1}\end{aligned}\quad (25)$$

为了减小计算舍入误差对滤波的影响，保持矩阵的对称正定性，一般改写成下面的形式：

$$P_k = (I - K_k H_k) P_{k,k-1} (I - K_k H_k)^T + K_k R_k K_k^T \quad (26)$$

卡尔曼滤波相对定位程序设计流程如下：

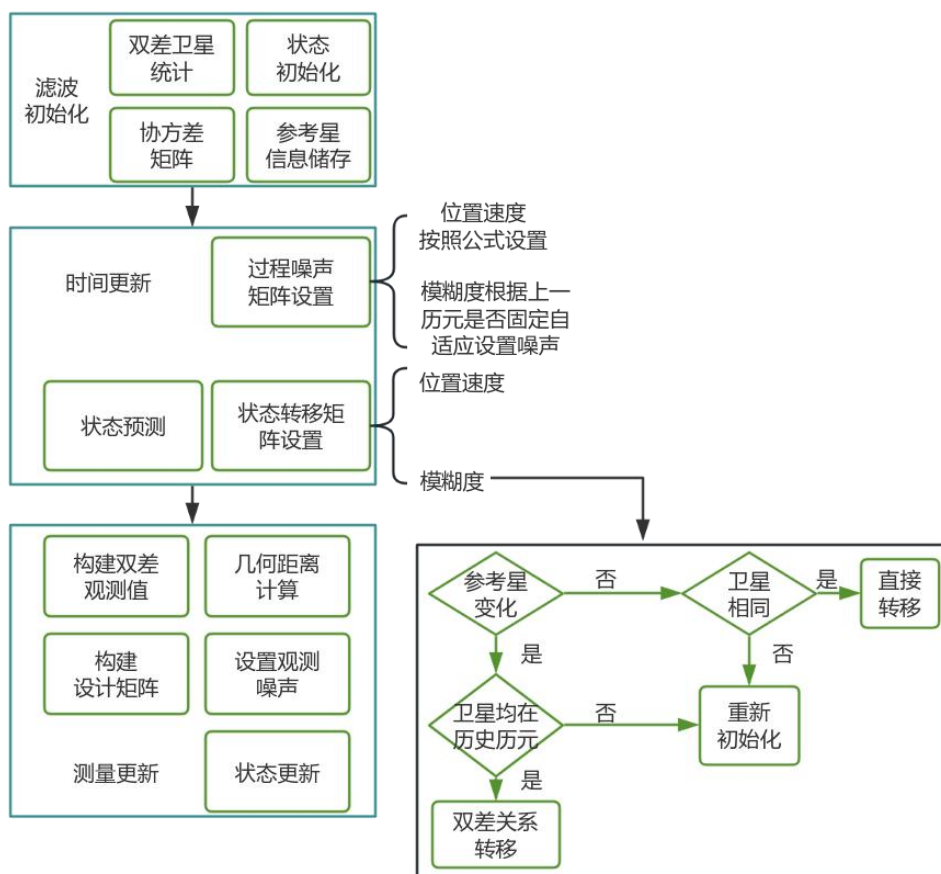


图 3 卡尔曼滤波浮点解程序设计

其中，模糊度状态转移矩阵的更新代码如下：

2.4 代码

状态转移矩阵程序设计

```
// 位置和速度不变
FAI.block(0, 0, NUM_OF_STATE_OF_NOAMU, NUM_OF_STATE_OF_NOAMU) = MatrixXd::Identity(NUM_OF_STATE_OF_NOAMU, NUM_OF_STATE_OF_NOAMU);
FAI.block(0, 3, 3, 3) = Matrix3d::Identity() * 1.0;
// 模糊度
vector<bool> need2init(num_amb, false);
for (int i = 0; i < new_sys_prn_pair.size(); i++)
{
    int n = new_sys_prn_pair[i].first, prn = new_sys_prn_pair[i].second;
    int new_rows = NUM_OF_STATE_OF_NOAMU + 2 * i;
    int ref_idx = getAmbIdx(sys_prn_pair, new_ref_prn[n], n);
    int idx = getAmbIdx(sys_prn_pair, prn, n);
    // 参考星相同
    if (new_ref_prn[n] == this->ref_prn[n])
```

```
{
    if (idx != -1) // 卫星存在于上历元
    {
        FAI(new_rows + 0, NUM_OF_STATE_OF_NOAMU + 2 * idx + 0) = 1.0;
        FAI(new_rows + 1, NUM_OF_STATE_OF_NOAMU + 2 * idx + 1) = 1.0;
    }
    else // 卫星新出现
    {
        need2init[2 * i] = true; need2init[2 * i + 1] = true;
    }
}
else if (ref_idx != -1 && idx != -1) // 参考星不同，但是当前历元的参考星存在于上历元
{
    FAI(new_rows + 0, NUM_OF_STATE_OF_NOAMU + 2 * idx + 0) = 1.0;
    FAI(new_rows + 0, NUM_OF_STATE_OF_NOAMU + 2 * ref_idx + 0) = -1.0;
    FAI(new_rows + 1, NUM_OF_STATE_OF_NOAMU + 2 * idx + 1) = 1.0;
    FAI(new_rows + 1, NUM_OF_STATE_OF_NOAMU + 2 * ref_idx + 1) = -1.0;
}
else // 其他情况，重新进行初始化
{
    need2init[2 * i] = true; need2init[2 * i + 1] = true;
}
}
```

2.4 代码

测量更新程序设计

```
void RTKEKF::update(rtkdata& rtkdata)
{
    int sum_num = rtkdata.DDObs.DDSatNum[0] + rtkdata.DDObs.DDSatNum[1];
    current_obs.Z = VectorXd::Zero(sum_num * 4), current_obs.R = MatrixXd::Zero(sum_num
    * 4, sum_num * 4), current_obs.H = MatrixXd::Zero(sum_num * 4, NUM_OF_STATE_OF_NO
    AMU + sum_num * 2);
    VectorXd Z_pred = VectorXd::Zero(sum_num * 4);
    // 构造观测值方差阵和基站到卫星的距离
    // 初始化基站和流动站位置
    MatrixXd Basepos(3, 1), Roverpos(3, 1);
    double norm_base = sqrt(pow(rtkdata.basepos.BestPos[0], 2) + pow(rtkdata.basepos.Bes
    tPos[1], 2) + pow(rtkdata.basepos.BestPos[2], 2));
    for (int i = 0; i < 3; i++)
    {
        if (norm_base == 0.0)
        {
            Basepos(i, 0) = rtkdata.basepos.Pos[i];
        }
    }
}
```

```

}
else
{
    Basepos(i, 0) = rtkdata.basepos.BestPos[i];
}
Roverpos(i, 0) = rtkdata.roverpos.Pos[i];
}
VectorXd Pbase2sat(sum_num);
double Psigma = 0.3, Lsigma = 0.01;
int rows = 0;
for (int i = 0; i < rtkdata.SdObs.SatNum; i++)
{
    int n = (rtkdata.SdObs.SdSatObs[i].System == GPS) ? 0 : 1;
    int ref_prn = rtkdata.DDObs.RefPrn[n], ref_idx = rtkdata.DDObs.RefPos[n];
    int rover_idx = rtkdata.SdObs.SdSatObs[i].nRov, base_idx = rtkdata.SdObs.SdSatObs[i].n
Bas;
    GNSSSys sys = rtkdata.SdObs.SdSatObs[i].System;
    // 跳过参考星/伪距和载波未通过周跳检测/卫星星历不正常/
    if (i == ref_idx || !rtkdata.SdObs.SdSatObs[i].Valid) continue;
    current_obs.Z(rows * 4 + 0) = rtkdata.SdObs.SdSatObs[i].dP[0] - rtkdata.SdObs.SdSatObs
[ref_idx].dP[0];
    current_obs.Z(rows * 4 + 1) = rtkdata.SdObs.SdSatObs[i].dP[1] - rtkdata.SdObs.SdSatObs
[ref_idx].dP[1];
    current_obs.Z(rows * 4 + 2) = rtkdata.SdObs.SdSatObs[i].dL[0] - rtkdata.SdObs.SdSatObs
[ref_idx].dL[0];
    current_obs.Z(rows * 4 + 3) = rtkdata.SdObs.SdSatObs[i].dL[1] - rtkdata.SdObs.SdSatObs
[ref_idx].dL[1];
    // 基站到所有非参考卫星的距离
    Pbase2sat(rows, 0) = sqrt(
        pow(Basepos(0) - rtkdata.base_obs.SatPVT[base_idx].SatPos[0], 2) +
        pow(Basepos(1) - rtkdata.base_obs.SatPVT[base_idx].SatPos[1], 2) +
        pow(Basepos(2) - rtkdata.base_obs.SatPVT[base_idx].SatPos[2], 2)
    );
    // 权阵初始化
    int cols = 0; // 里层有效卫星数
    for (int j = 0; j < rtkdata.SdObs.SatNum; j++)
    {
        int rover_idx_tmp = rtkdata.SdObs.SdSatObs[j].nRov, base_idx_tmp = rtkdata.SdObs.Sd
SatObs[j].nBas;
        if (j == rtkdata.DDObs.RefPos[0] || j == rtkdata.DDObs.RefPos[1] || !rtkdata.SdObs.SdSat
Obs[j].Valid) continue;
        if (sys != rtkdata.SdObs.SdSatObs[j].System) {
            cols++; continue; // 卫星有效但系统不同
        }
    }
}

```

```

int ndd = (sys == GPS) ? rtkdata.DDObs.DDSatNum[0] : rtkdata.DDObs.DDSatNum[1];
double a = (rows == cols) ? 4 : 2;
current_obs.R(rows * 4 + 0, cols * 4 + 0) = a * (Psigma * Psigma);
current_obs.R(rows * 4 + 1, cols * 4 + 1) = a * (Psigma * Psigma);
current_obs.R(rows * 4 + 2, cols * 4 + 2) = a * (Lsigma * Lsigma);
current_obs.R(rows * 4 + 3, cols * 4 + 3) = a * (Lsigma * Lsigma);
cols++;
}
rows++;
}
Vector3d baseref_sat_PVT_GPS, baseref_sat_PVT_BDS, roverref_sat_PVT_GPS, roverref_sat_PVT_BDS;
// 获取设计矩阵
int ref_in_base[2] = { rtkdata.SdObs.SdSatObs[rtkdata.DDObs.RefPos[0]].nBas,
    rtkdata.SdObs.SdSatObs[rtkdata.DDObs.RefPos[1]].nBas };
for (int i = 0; i < 3; i++)
{
    baseref_sat_PVT_GPS(i, 0) = rtkdata.base_obs.SatPVT[ref_in_base[0]].SatPos[i];
    baseref_sat_PVT_BDS(i, 0) = rtkdata.base_obs.SatPVT[ref_in_base[1]].SatPos[i];
}
double Pbase2sat_ref[2] = {
    (Basepos - baseref_sat_PVT_GPS).norm(),
    (Basepos - baseref_sat_PVT_BDS).norm()
};
int ref_in_rover[2] = { rtkdata.SdObs.SdSatObs[rtkdata.DDObs.RefPos[0]].nRov,
    rtkdata.SdObs.SdSatObs[rtkdata.DDObs.RefPos[1]].nRov };
VectorXd Prover2sat(sum_num);
for (int i = 0; i < 3; i++)
{
    roverref_sat_PVT_GPS(i, 0) = rtkdata.rover_obs.SatPVT[ref_in_rover[0]].SatPos[i];
    roverref_sat_PVT_BDS(i, 0) = rtkdata.rover_obs.SatPVT[ref_in_rover[1]].SatPos[i];
}
double Prover2sat_ref[2] = {
    (current_state.X.head(3) - roverref_sat_PVT_GPS).norm(),
    (current_state.X.head(3) - roverref_sat_PVT_BDS).norm()
};
rows = 0;
for (int i = 0; i < rtkdata.SdObs.SatNum; i++)
{
    int n = (rtkdata.SdObs.SdSatObs[i].System == GPS) ? 0 : 1;
    int ref_prn = rtkdata.DDObs.RefPrn[n], ref_idx = rtkdata.DDObs.RefPos[n];
    int rover_idx = rtkdata.SdObs.SdSatObs[i].nRov, base_idx = rtkdata.SdObs.SdSatObs[i].nBas;
    if (i == ref_idx || !rtkdata.SdObs.SdSatObs[i].Valid) continue;

```



```

Prover2sat(rows) = sqrt(pow(current_state.X(0) - rtkdata.rover_obs.SatPVT[rover_idx].SatPos[0], 2) +
    pow(current_state.X(1) - rtkdata.rover_obs.SatPVT[rover_idx].SatPos[1], 2) +
    pow(current_state.X(2) - rtkdata.rover_obs.SatPVT[rover_idx].SatPos[2], 2));
double lamuda1 = 0, lamuda2 = 0;
if (rtkdata.SdObs.SdSatObs[i].System == GPS) { lamuda1 = WL1_GPS; lamuda2 = WL2_GPS; }
else { lamuda1 = WL1_BDS; lamuda2 = WL3_BDS; }
// 设计矩阵
double l_ = (current_state.X(0) - rtkdata.rover_obs.SatPVT[rover_idx].SatPos[0]) / Prover2sat(rows)
    - (current_state.X(0) - rtkdata.rover_obs.SatPVT[ref_in_rover[n]].SatPos[0]) / Prover2sat_ref[n];
double m_ = (current_state.X(1) - rtkdata.rover_obs.SatPVT[rover_idx].SatPos[1]) / Prover2sat(rows)
    - (current_state.X(1) - rtkdata.rover_obs.SatPVT[ref_in_rover[n]].SatPos[1]) / Prover2sat_ref[n];
double n_ = (current_state.X(2) - rtkdata.rover_obs.SatPVT[rover_idx].SatPos[2]) / Prover2sat(rows)
    - (current_state.X(2) - rtkdata.rover_obs.SatPVT[ref_in_rover[n]].SatPos[2]) / Prover2sat_ref[n];
for (int j = 0; j < 4; j++)
{
    current_obs.H(4 * rows + j, 0) = l_; current_obs.H(4 * rows + j, 1) = m_; current_obs.H(4 * rows + j, 2) = n_;
}
Z_pred(4 * rows + 0) = Prover2sat(rows) - Pbase2sat(rows) - Prover2sat_ref[n] + Pbase2sat_ref[n];
Z_pred(4 * rows + 1) = Prover2sat(rows) - Pbase2sat(rows) - Prover2sat_ref[n] + Pbase2sat_ref[n];
Z_pred(4 * rows + 2) = Prover2sat(rows) - Pbase2sat(rows) - Prover2sat_ref[n] + Pbase2sat_ref[n] + lamuda1 * current_state.X(NUM_OF_STATE_OF_NOAMU + 2 * rows + 0);
Z_pred(4 * rows + 3) = Prover2sat(rows) - Pbase2sat(rows) - Prover2sat_ref[n] + Pbase2sat_ref[n] + lamuda2 * current_state.X(NUM_OF_STATE_OF_NOAMU + 2 * rows + 1);
current_obs.H(rows * 4 + 2, NUM_OF_STATE_OF_NOAMU + rows * 2 + 0) = lamuda1;
current_obs.H(rows * 4 + 3, NUM_OF_STATE_OF_NOAMU + rows * 2 + 1) = lamuda2;
rows++;
}
// 增益矩阵
Kk = current_state.P * current_obs.H.transpose() * (current_obs.H * current_state.P * current_obs.H.transpose() + current_obs.R).inverse();
// 新息
Vkk1 = current_obs.Z - Z_pred;
// 状态滤波和滤波方差

```

```

current_state.X = current_state.X + Kk * Vkk1;
current_state.P = (MatrixXd::Identity(current_state.P.rows(), current_state.P.cols()) - Kk * current_obs.H) * current_state.P * (MatrixXd::Identity(current_state.P.rows(), current_state.P.cols()) - Kk * current_obs.H).transpose() + Kk * current_obs.R * Kk.transpose();
//模糊度固定
rtkdata.DDObs.bFixed = false;
rtkdata.DDObs.Ratio = 0.0;
int n = sum_num * 2; // 模糊度参数个数
int noamb_param_num = 6; // 非模糊度参数个数 (位置速度参数)
int total_params = 6 + n;
//计算协方差矩阵
MatrixXd float_Qxx;
float_Qxx = current_state.P;
// 检查是否有足够的模糊度进行固定
if (n > 0 && float_Qxx.rows() >= total_params)
{
    // 浮点模糊度和协方差
    VectorXd float_ambiguities = current_state.X.segment(noamb_param_num, n);
    VectorXd fixed_ambiguities(n);
    MatrixXd Q_ambiguities = float_Qxx.block(noamb_param_num, noamb_param_num, n, n);
    VectorXd X_float = current_state.X.head(3);
    MatrixXd Q_pos_amb = float_Qxx.topRightCorner(noamb_param_num-3, n);
    // 调用 LAMBDA 算法
    double* a_ptr = float_ambiguities.data();
    double* Q_ptr = Q_ambiguities.data();
    double* s_ptr = rtkdata.DDObs.ResAmb;
    if (lambda(n, rtkdata.DDObs.m, a_ptr, Q_ptr, rtkdata.DDObs.FixedAmb, rtkdata.DDObs.ResAmb) == 0)
    {
        // 模糊度确认
        rtkdata.DDObs.Ratio = rtkdata.DDObs.ResAmb[1] / rtkdata.DDObs.ResAmb[0];
        if (rtkdata.DDObs.Ratio >= 3.0)
        {
            // 固定成功, 计算固定解
            rtkdata.DDObs.bFixed = true;
            for (int i = 0; i < float_ambiguities.size(); i++)
            {
                fixed_ambiguities(i) = rtkdata.DDObs.FixedAmb[i];
            }
            VectorXd X_fixed = X_float - Q_pos_amb * Q_ambiguities.inverse() * (float_ambiguities - fixed_ambiguities);
            // 更新流动站位置为固定解
            current_state.X.head(3) = X_fixed;
        }
    }
}

```

```

current_state.P.block(0, 0, noamb_param_num, noamb_param_num) -= float_Qxx.topRightCorner(noamb_param_num, n) * Q_ambiguities.inverse() * float_Qxx.topRightCorner(noamb_param_num, n).transpose();
for (int k = 0; k < n; k++)
{
    int idx = noamb_param_num + k;
    current_state.X(idx) = round(fixed_ambiguities(k)); // 模糊度转换成整数
    current_state.P(idx, idx) = 1e-10;
    for (int j = 0; j < current_state.P.cols(); j++)
    {
        if (j != idx)
        {
            current_state.P(idx, j) = 0.0;
            current_state.P(j, idx) = 0.0;
        }
    }
}
}
}
if (!rtkdata.DDObs.bFixed)
{
    // 使用浮点解
    current_state.X.head(3) = X_float;
}
else
{
    // 没有足够卫星进行模糊度固定, 使用浮点解
    cout << "卫星数量不足" << endl;
}
}

```

2.5. 模糊度固定

LAMBDA (Least-squares AMBiguity Decorrelation Adjustment) 方法是一种广泛应用于 GNSS 高精度定位中的整数模糊度固定技术, 该方法通过对浮点模糊度及其协方差矩阵进行整数最小二乘估计和降相关处理, 有效缩小搜索空间, 并采用高效的搜索策略快速确定最优的

整数模糊度组合，从而将载波相位观测值的浮点解提升为固定解，显著提高相对定位的精度至厘米级。

当给定两个实数模糊度 $\hat{N}_1$ 、 $\hat{N}_2$ ，设其协因数阵： $Q_{\hat{N}} = \begin{bmatrix} q_{\hat{N}_1\hat{N}_1} & 0 \\ 0 & q_{\hat{N}_2\hat{N}_2} \end{bmatrix}$,

模糊度误差不相关，则

$$\begin{aligned} \chi^2(N) &= (\hat{N} - N)^T Q_{\hat{N}}^{-1} (\hat{N} - N) = \min \\ \Rightarrow \chi^2(N) &= \frac{(\hat{N}_1 - N_1)^2}{q_{\hat{N}_1\hat{N}_1}} + \frac{(\hat{N}_2 - N_2)^2}{q_{\hat{N}_2\hat{N}_2}} = \min \end{aligned} \quad (27)$$

当 N_1 、 N_2 为最接近实数值的整数值时，可满足上式的条件。在 $N_1 - N_2$ 坐标系中表示一个椭圆，这个椭圆通常被认为是浮点模糊度的搜索空间。

但实际上，模糊度是相关的， $Q_{\hat{N}} = \begin{bmatrix} q_{\hat{N}_1\hat{N}_1} & q_{\hat{N}_1\hat{N}_2} \\ q_{\hat{N}_2\hat{N}_1} & q_{\hat{N}_2\hat{N}_2} \end{bmatrix}$, $q_{\hat{N}_1\hat{N}_2} \neq 0$ ，此时对应的椭圆是相对于 $N_1 - N_2$ 坐标系产生了一个旋转的椭圆。此时搜索满足条件的 N_1 、 N_2 将变得更为复杂。因此需要采用模糊度降相关方法降低模糊度之间的相关性，使得变换后的模糊度方差阵恢复为对角阵。在观测时间段比较短的情况下，模糊度参数实数解的精度低，且相关性很强；这是造成模糊度解算困难的主要原因。保证超椭圆（椭球）容积不变的条件下通过整数变化降低相关性，对角占优，非对角线元素小于等于 0.5，同时降低模糊度参数的方差，提高精度。经过整数变换后的参数不会改变原始参数的整数特性，并且是可逆的。从这个角度，整数变换是完全独立于搜索过程的。

进行最小二乘或卡尔曼滤波解算时，可以得到基线分量和浮点模

模糊度，采用 Z 变换，对模糊度搜索空间进行重新参数化，降低浮点模糊度之间的相关性（整数变换降相关），通过使用序贯条件最小二乘平差和一个离散搜索策略来估算整数模糊度。通过逆变换 Z^{-1} 将模糊度重新转换回原来的模糊度空间，将整数模糊度作为整数固定，确定最终的基线分量。LAMBDA 算法流程如下图所示：

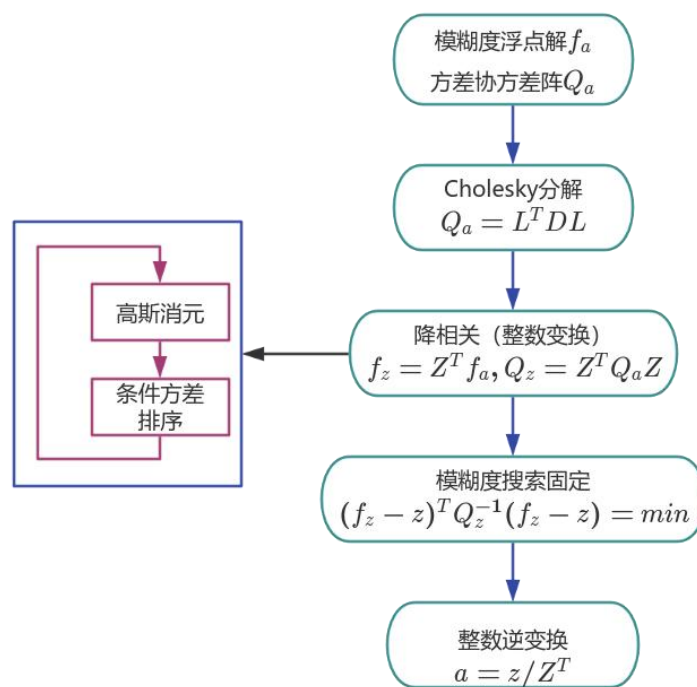


图 4 LAMBDA 算法流程

一般通过 ration 统计检验确认模糊度最优解，即整数解中最小单位权中误差与次最小单位权中误差间的显著性检验，阈值设置为 3.0, $\text{ratio} = \frac{\sigma_{\text{次最小}}^2}{\sigma_{\text{最小}}^2} \geq \zeta_{F\left(\mu, f, 1-\frac{\alpha}{2}\right)}$ 。并将最优整数模糊度组合代回原法方程平差计算，得出基线向量解和方差阵。若基线向量浮点解和模糊度浮点解为 $\hat{b}$ 和 $\hat{a}$ ，基线向量固定解和模糊度固定解为 $\check{b}$ 和 $\check{a}$ 。则固定解基线

向量求解为

$$\tilde{b} = \hat{b} - Q_{\hat{b}\hat{a}} Q_{\hat{a}}^{-1} (\hat{a} - \tilde{a}) \quad (28)$$

LAMBDA 固定方法有开源的软件可以使用，本软件中使用老师提供的版本。卡尔曼滤波和最小二乘的模糊度固定流程基本相同，唯一的不同在卡尔曼滤波浮点解固定后需要保存整周模糊度和更新后的协方差阵，下面以卡尔曼滤波的模糊度固定流程为例。

2.5 代码

模糊度固定程序设计

```
//模糊度固定
rtkdata.DDObs.bFixed = false;
rtkdata.DDObs.Ratio = 0.0;
int n = sum_num * 2; // 模糊度参数个数
int noamb_param_num = 6; // 非模糊度参数个数（位置速度参数）
int total_params = 6+n;
//计算协方差矩阵
MatrixXd float_Qxx;
float_Qxx = current_state.P;
// 检查是否有足够的模糊度进行固定
if (n > 0 && float_Qxx.rows() >= total_params)
{
    // 浮点模糊度和协方差
    VectorXd float_ambiguities = current_state.X.segment(noamb_param_num, n);
    VectorXd fixed_ambiguities(n);
    MatrixXd Q_ambiguities = float_Qxx.block(noamb_param_num, noamb_param_num, n, n);
    VectorXd X_float = current_state.X.head(3);
    MatrixXd Q_pos_amb = float_Qxx.topRightCorner(noamb_param_num-3, n);
    // 调用 LAMBDA 算法
    double* a_ptr = float_ambiguities.data();
    double* Q_ptr = Q_ambiguities.data();
    double* s_ptr = rtkdata.DDObs.ResAmb;
    if (lambda(n, rtkdata.DDObs.m, a_ptr, Q_ptr, rtkdata.DDObs.FixedAmb, rtkdata.DDObs.ResAmb) == 0)
    {
        // 模糊度确认
        rtkdata.DDObs.Ratio = rtkdata.DDObs.ResAmb[1] / rtkdata.DDObs.ResAmb[0];
        if (rtkdata.DDObs.Ratio >= 3.0)
        {

```

```
// 固定成功, 计算固定解
rtkdata.DDObs.bFixed = true;
for (int i = 0; i < float_ambiguities.size(); i++)
{
    fixed_ambiguities(i) = rtkdata.DDObs.FixedAmb[i];
}
VectorXd X_fixed = X_float - Q_pos_amb * Q_ambiguities.inverse() * (float_ambiguities
- fixed_ambiguities);
// 更新流动站位置为固定解
current_state.X.head(3) = X_fixed;
current_state.P.block(0, 0, noamb_param_num, noamb_param_num) -= float_Qxx.topRightCorner(noamb_param_num, n) * Q_ambiguities.inverse() * float_Qxx.topRightCorner(noamb_param_num, n).transpose();
for (int k = 0; k < n; k++)
{
    int idx = noamb_param_num + k;
    current_state.X(idx) = round(fixed_ambiguities(k)); // 模糊度转换成整数
    current_state.P(idx, idx) = 1e-10;
    for (int j = 0; j < current_state.P.cols(); j++)
    {
        if (j != idx)
        {
            current_state.P(idx, j) = 0.0;
            current_state.P(j, idx) = 0.0;
        }
    }
}
if (!rtkdata.DDObs.bFixed)
{
    // 使用浮点解
    current_state.X.head(3) = X_float;
}
else
{
    // 没有足够卫星进行模糊度固定, 使用浮点解
    cout << "卫星数量不足" << endl;
}
```

3. 结果精度分析

3.1. 数据文件说明

软件输出的定位结果包括 GPS 周，GPS 周内秒，流动站的地心地固坐标系的 X、Y、Z 坐标值，是否是固定解，ratio 值，表头信息如下图所示：

A	B	C	D	E	F	G	H
week	sec	X	Y	Z	fixed	ratio	
2390	466894	-2267815	5009335	3221024	1	3.6528	
2390	466895	-2267815	5009335	3221024	1	3.9038	
2390	466896	-2267815	5009335	3221024	1	3.5542	
2390	466897	-2267815	5009335	3221024	1	3.8554	
2390	466898	-2267815	5009335	3221024	1	3.8314	
2390	466899	-2267815	5009335	3221024	1	3.8185	
2390	466900	-2267815	5009335	3221024	1	3.8629	
2390	466901	-2267815	5009335	3221024	1	3.8324	
2390	466902	-2267815	5009335	3221024	1	3.6239	
2390	466903	-2267815	5009335	3221024	1	3.696	

图 5 结果文件信息（部分）

报告中一共使用了四个结果文件，分别是：20 度最小高度角阈值的最小二乘相对定位结果、20 度最小高度角阈值的卡尔曼滤波相对定位结果、无高度角限制的最小二乘相对定位结果、无高度角限制的卡尔曼滤波相对定位结果。

3.2. 定位结果

通过 MATLAB 软件读取 csv 结果文件的数据，计算出 20h 定位结果的均值和精度指标，最小二乘定位结果和滤波定位结果如下表所示：

表 1 定位结果

指标 方法	X/m	Y/m	Z/m	固定解 RMS	浮点解 RMS	固定率/%
最小二乘	-2267815.0772	5009335.3956	3221024.0135	0.019	0.588	95.81
滤波	-2267815.0771	5009335.3955	3221024.0134	0.019	0.626	95.79

通过分析指标可以发现，最小二乘和卡尔曼滤波的定位结果差距很小，相差基本在 0.1mm 左右，这对于精密单点定位来说可以认为

两者定位结果是相同的。两种方法的固定解均方根误差基本也没有差距，这说明在模糊度都能固定的情况下，两种定位结果的方法性能都较好。但是模糊度不能固定的时候，滤波的定位结果波动相较于最小二乘方法的波动更大，且固定率略低于最小二乘方法。通过这些指标可以看出，设计的卡尔曼滤波仍有缺陷，即没有将固定历元的信息传递下去，使得固定率提高，而且也没有提高浮点解的精度。

3.3. 定位误差分析

以定位结果坐标的均值作为坐标参考真值，将定位结果转换到以参考真值为中心的站心坐标系下，该部分分析了两种定位方法的定位误差。两种方法的 **ENU** 坐标系的定位误差如下图所示：

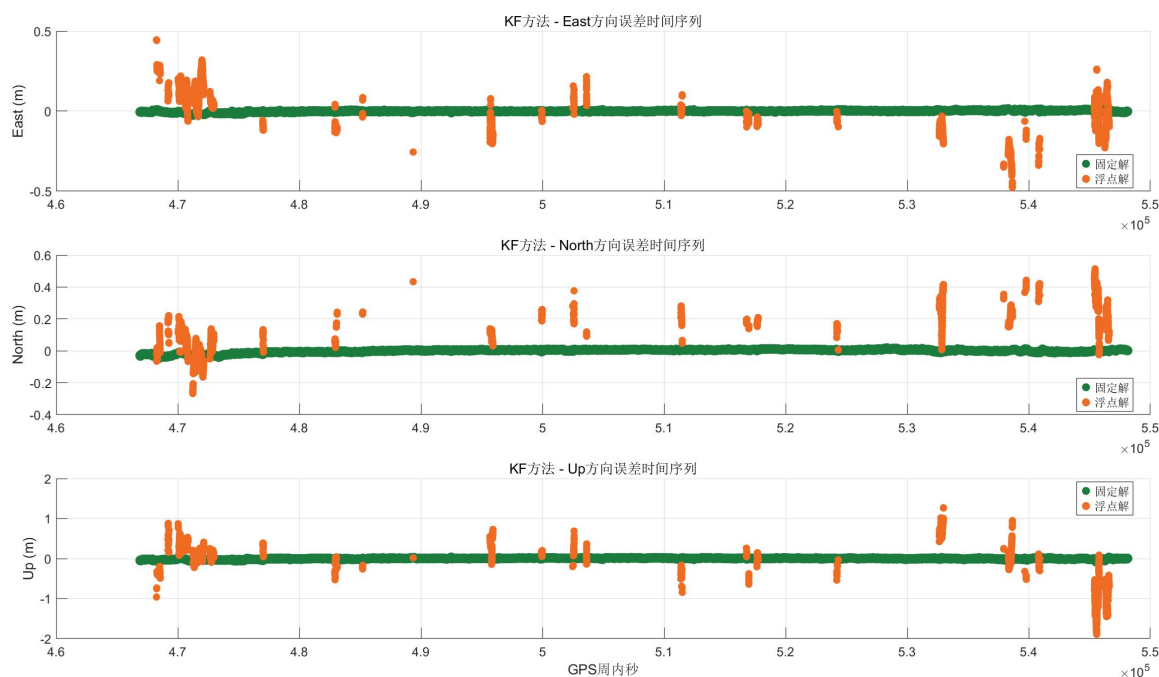


图 6 卡尔曼滤波相对定位 ENU 误差

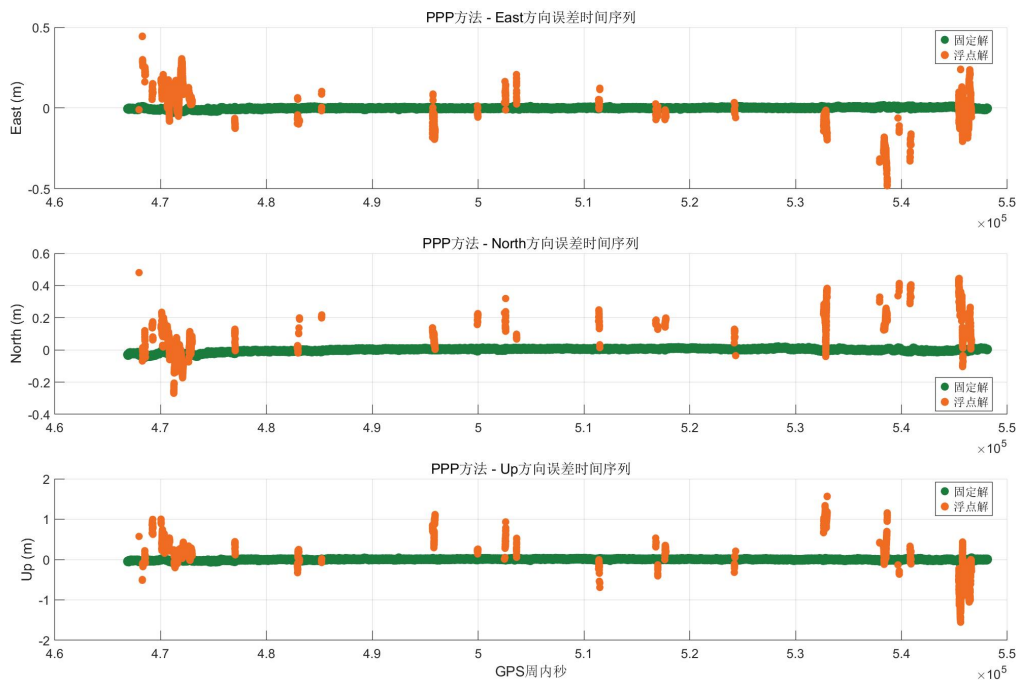


图 7 最小二乘相对定位 ENU 误差

从 ENU 误差时间序列图中可以看出，精密单点定位的固定解误差非常小，符合毫米级定位精度的认识。浮点解水平方向上的误差也能控制在分米级，优于标准单点定位；天顶误差达米级，与标准单点定位的误差水平一致。对比两种定位方法发现，最小二乘相对定位的浮点解在天顶方向上的误差要略差于卡尔曼滤波。

两种定位方法的误差三维散点图如下图所示：

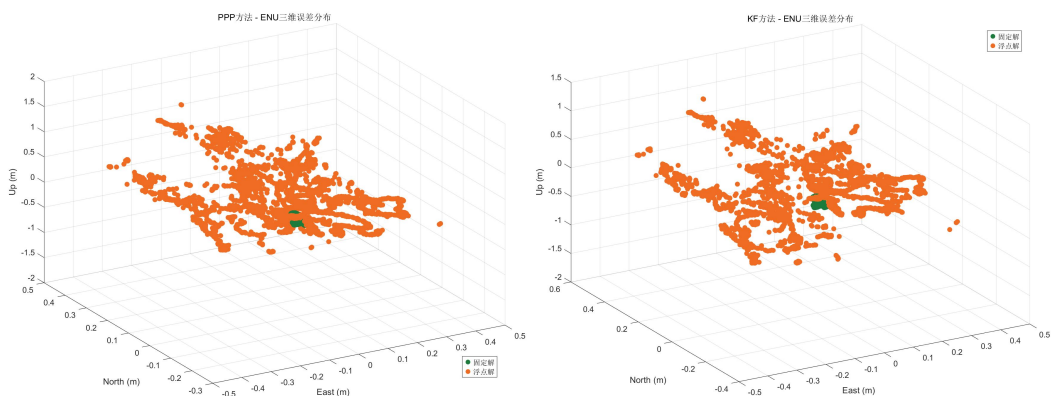


图 8 ENU 误差三维散点图

图 8 显示，滤波的定位浮点解结果在天顶方向的最大误差要优于

最小二乘方法，但是最小二乘得到的浮点解会更收敛。可能是由于设计的滤波没有正确地传递固定解信息，但是浮点解的信息却被后续的历元使用了，导致长时间未固定的历元定位结果反而更差。卡尔曼滤波和最小二乘定位结果水平误差如下图所示：



图 9 卡尔曼滤波相对定位水平位置误差

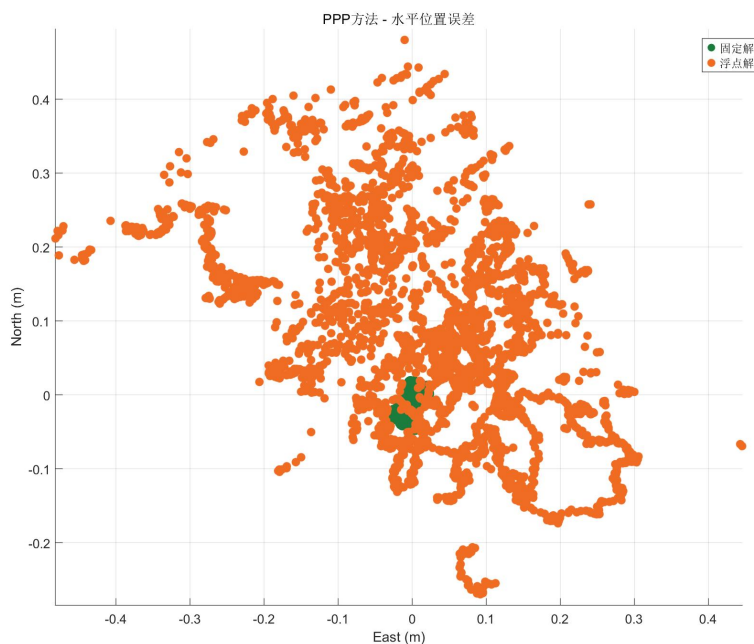


图 10 最小二乘相对定位水平位置误差

图 9 和图 10 显示，卡尔曼滤波和最小二乘在固定解的误差水平上

相近，但是最小二乘在浮点解上表现更优秀，误差更小。

为了研究两种定位方法定位结果的质量，下面给出两种定位方法各自的 **Ratio** 时间序列图：

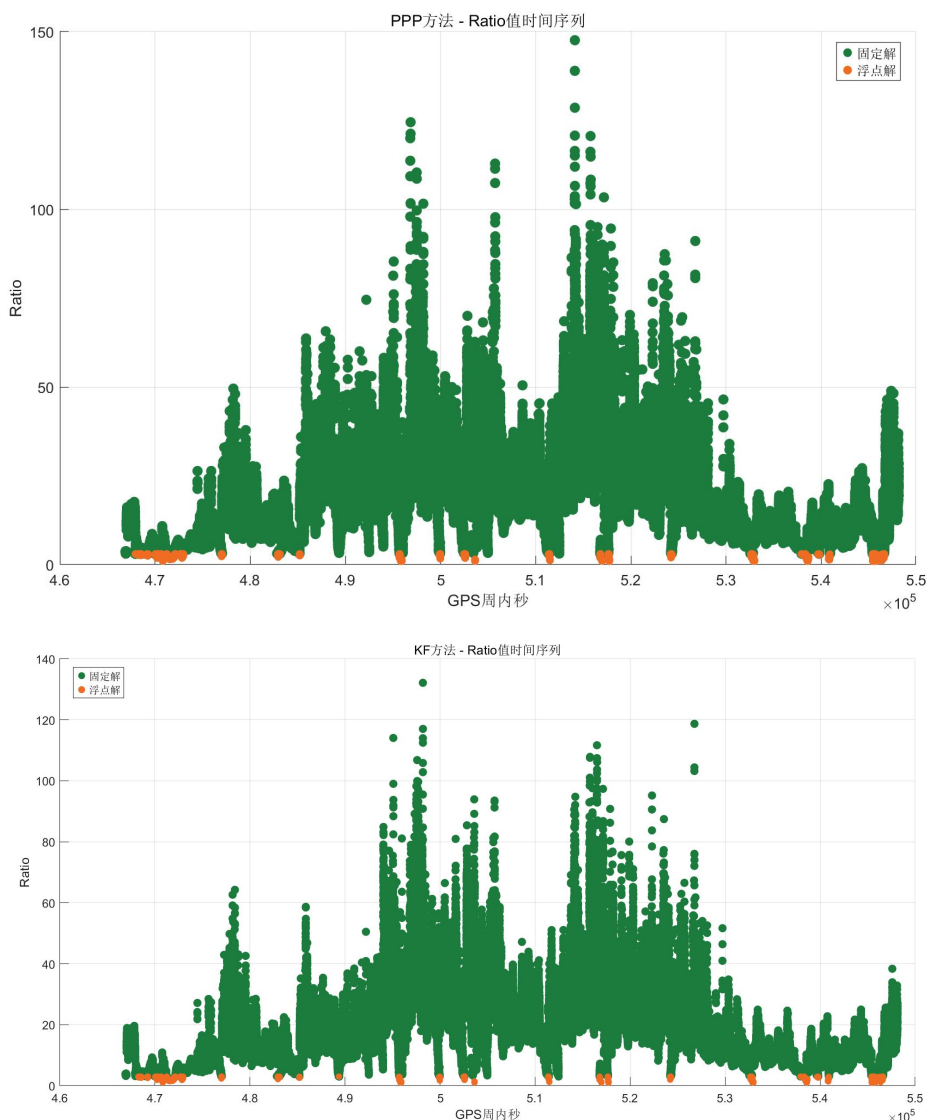


图 11 Ratio 时间序列对比

从整体趋势上看，两种定位方法的 **Ratio** 值变化相近，最小二乘相对定位计算得到的 **Ratio** 值均值为 18.06，卡尔曼滤波计算得到的 **Ratio** 均值为 18.11，相差较小，说明两者在定位结果的质量都比较高。其中，**Ratio** 值与定位误差存在一定的关系，为了研究这一关系，绘制出 **Ratio** 与水平误差的散点图如下：

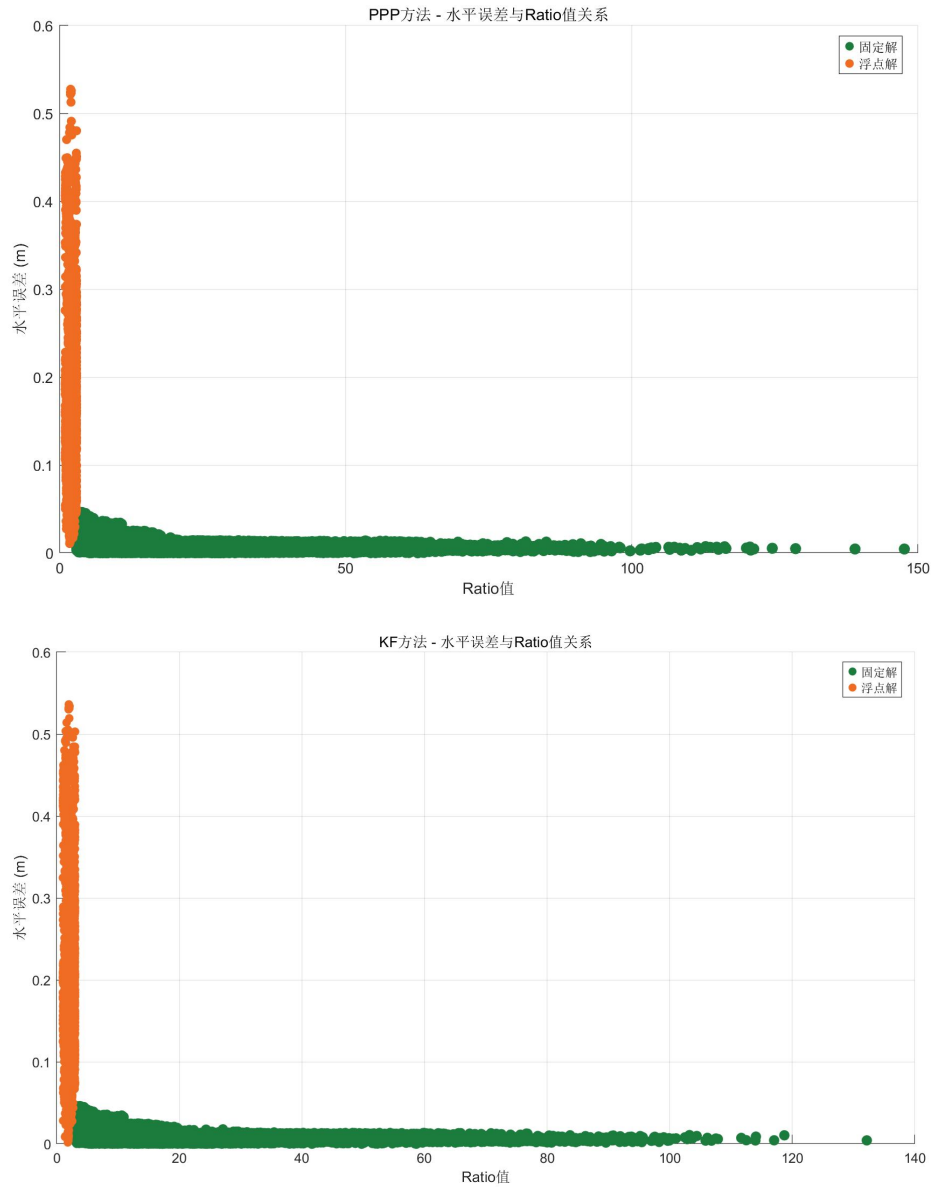


图 12 水平误差与 Ratio 值的关系

图 12 表明，水平误差和 Ratio 值存在反比例关系，当 Ratio 值增大时，对应的水平误差也在减少，因此也可以通过 Ratio 值去间接评估定位结果的质量。

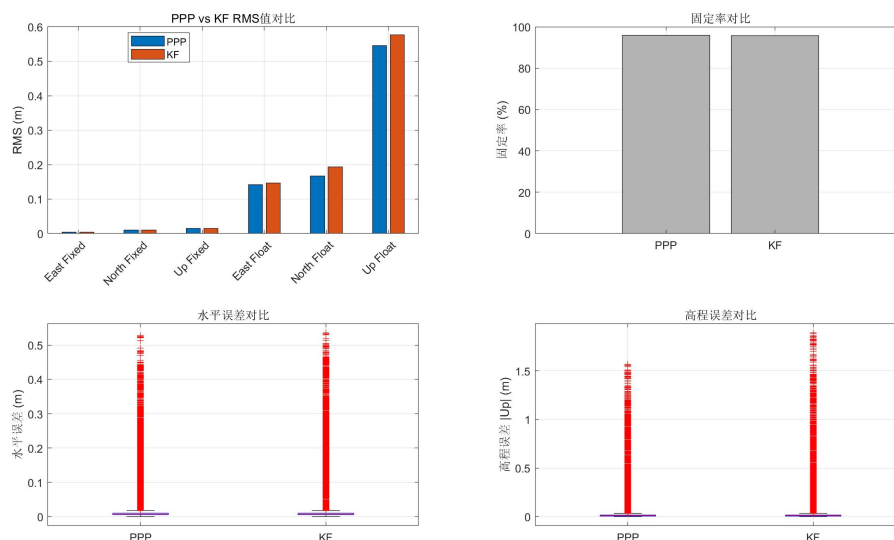


图 13 卡尔曼滤波和最小二乘综合对比

图 13 表明，最小二乘和卡尔曼滤波在固定解上 RMS 相当，但是最小二乘的浮点解误差要更小；在水平误差上，两者也是相当；但是高程误差中，最小二乘定位方法的离群点的值更小，说明定位效果更好。

在设计程序的过程中，我曾经漏掉了检查卫星高度角的模块，导致低高度角卫星也参与了解算，发现这些卫星虽然增加了冗余观测值，但是低质量的观测数据导致固定率下降，但是解算精度却提高了，现将定位结果统计在下表中：

表 2 未剔除低高度角卫星定位结果

指标 方法	X/m	Y/m	Z/m	固定 解 RMS	浮点 解 RMS	固定 率/%
最小二乘	-2267815.0775	5009335.3960	3221024.0140	0.016	0.492	87.55
滤波	-2267815.0774	5009335.3959	3221024.0139	0.016	0.454	87.69

此数据与表 1 的精度指标对比发现，除了固定率下降之外，RMS 指标反而提升了。这是因为，虽然低高度角卫星的低质量观测数据会为定位结果带来误差，但是冗余观测值会为平差结果起到更加积极的

作用。

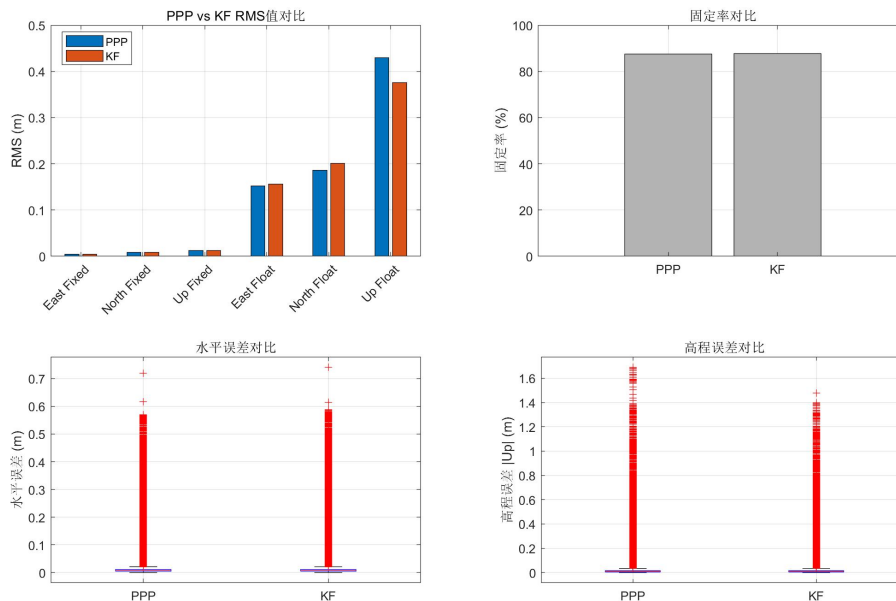


图 14 未剔除低高度角卫星的两种定位方法对比

不难发现，在未剔除低高度角卫星的情况下，尽管 RMS 有所下降，但是定位的误差水平增加了，这说明这些低质量的观测卫星仍然对定位效果有着负面效果，且影响了定位的连续性，在卫星数量较多的情况下，应当考虑剔除这些卫星。

4. 体会与心得

通过本次“卫星导航算法与程序设计 2”课程的实践，我完整地实现了 RTK 相对定位算法，从数据解码、时间同步、单差与双差观测值构建，到最小二乘与卡尔曼滤波两种解算方法的实现，再到 LAMBDA 模糊度固定与结果分析，全面加深了对 GNSS 高精度定位理论的理解，也提升了算法实现与系统调试的能力。

程序设计过程中，我遇到的最大的问题就是调试一个错误的时间非常长。在面试的时候王老师您说，写程序之前要仔仔细细地把整个

算法流程推演一遍，就不容易出错了。后来我回去重新写卡尔曼滤波相对定位的时候，又结合课件重新把整个算法推导了一遍，很快就把程序改对了。您总是在课堂上强调，只有把算法流程真正掌握了，才能写出正确的程序。我以前总觉得这样非常浪费时间，所以每次粗略地过一遍 PPT 后就开始着手编程，但事实是调试程序找错误往往会花费更多时间。

其次就是一些细节部分的错误，虽然暂时没有在程序中体现出来，但是后续一定会出错。您在面试中给我指出的那个判断语句的错误，其实我在写下的时候完全没有意识这一个简单的语句在后面可能会出现 bug。这让我认识到，将课本上的理论知识运用到实践这个过程中，会遇到非常多的问题，一方面需要我们去试错积累经验，另一方面也很需要老师们多年实践积累下来的经验。如果有机会，我也想跟后来的学弟学妹说，上课一定要认真听讲，写程序之前一定要认真思考。

我还记得两个学期的课程，每次去面试的时候，王老师您第一句话都是：“感觉怎么样啊，有没有感觉自己的编程能力比以前强很多了？”但是我每次都不敢直面这个问题，因为我觉得我完成这个课程的程序都是磕磕绊绊的，面对更复杂的程序肯定是很难完成。但是当我这个学期初次接触科研时，师姐给了我很多代码，让我整合一下完成某项工作时，我发现我不像以前那样束手无措了，能够有条理地一点点完成任务。我觉得这门课程真的让我成长了很多，解决问题的能力也比以前更强，谢谢王老师的指导！